

ECE6140 Project: Combinational Circuit Simulator

1. Data Structures (one page max)

The simulator models a combinational circuit using the following core abstractions:

Gate (struct/class): represents a logic element. Each gate stores:

- **gate_name**: unique identifier (e.g., `and_n10`).
- **gate_type**: enum over `INPUT`, `OUTPUT`, `BUF`, `NOT`, `AND`, `OR`, `NAND`, `NOR`, `XOR`, `XNOR`.
- **input_number**: arity derived from the netlist line.
- **gate_input_values**: vector of symbolic values `GateValue`.
- **gate_output_value**: a `GateValue` in $\{0, 1, X, D, D'\}$. (This enables five-valued logic; for this assignment we drive only 0/1.)
- Fan-in/out: **fan_in_gates** and **fan_out_gates** lists to traverse the graph.

GateValue (enum): `ZERO`, `ONE`, `UNKNOWN`, `D`, `D_BAR` with overloaded logical operators (`&`, `|`, `~`, `^`) to evaluate multi-valued logic consistently across gates.

Wire (struct/class): names a net in the circuit and binds producers/consumers.

- **wire_name**: net label from the netlist.
- **input_gate**: the single driver gate for this net.
- **output_gates**: list of sinks receiving the net as input.

Circuit (container): owns all objects and provides APIs.

- Maps: **wires**: `map<string, Wire>`, **gates**: `map<string, Gate>`.
- Port order: **input_port_names** and **output_port_names** (stable ordering for vector I/O).
- Build: **add_a_gate_with_wires(...)** creates gates, ensures wires exist, and records fan-in/out via wires.
- Connect: **connect_gates()** walks wires to connect producers to consumers.
- Evaluate/reset: **evaluate()** performs a forward traversal; **reset_gates()** clears state between vectors.

CircuitWrapper: thin helper to parse a netlist file (e.g., `s27.txt`) and populate a `Circuit`. It also provides `evaluate_circuit_with_normal_input(bitstring)` which:

1. Resets all gate values to UNKNOWN.
2. Assigns ZERO/ONE to each INPUT gate based on the input bitstring order.
3. Calls `Circuit.evaluate()` then collects OUTPUT gate values in the declared output order.

This design keeps the netlist graph explicit (via wires) and enables deterministic vector I/O ordering required for tabulated results.

2. Simulation Algorithm (pseudocode)

The simulator uses event-driven forward propagation from inputs. Pseudocode below summarizes the core flow:

```
function evaluate_circuit_with_normal_input(input_bits):
    circuit.reset_gates()
    assert len(input_bits) == len(circuit.input_port_names)
    for i, input_name in enumerate(circuit.input_port_names):
        gate = circuit.gates[input_name]
        gate.set_output_value(ONE if input_bits[i] == '1' else ZERO)

    # Initialize evaluation queue with immediate fanout of inputs
    Q = deque()
    for input_name in circuit.input_port_names:
        Q.extend(circuit.gates[input_name].fan_out_gates)

    while Q not empty:
        g = Q.pop_left()
        prev = g.gate_output_value
        new = g.forward()          # reads fan-in outputs, applies gate logic
        if prev == new:
            continue              # stabilized; skip re-enqueuing
        assert prev == UNKNOWN    # acyclic assumption for these benchmarks
        for h in g.fan_out_gates:
            Q.append(h)

    # Read outputs in the netlist-declared order
    out_bits = []
    for out_name in circuit.output_port_names:
        v = circuit.gates[out_name].gate_output_value
        out_bits.append('1' if v == ONE else '0' if v == ZERO else 'X')
    return ''.join(out_bits)
```

3. Simulation Results (inputs and outputs)

All vectors below were produced by running the provided implementation on the four benchmark circuits. Each table shows input and output bit-vectors in order.

s27

Input (7 bits)	Output (4 bits)
1110101	1001
0001010	0100
1010101	1001
0110111	0001
1010001	1001

s298f_2

Input (17 bits)	Output (20 bits)
10101010101010101	00000010101000111000
01011110000000111	00000000011000001000
11111000001111000	00000000001111010010
11100001110001100	00000000100100100101
01111011110000000	11111011110000101101

s344f_2

Input (24 bits)	Output (26 bits)
101010101010101011111111	10101010101010101010101101
010111100000001110000000	00011110000000100001111100
111110000011110001111111	00011100000111011000111010
111000011100011000000000	00001101111001111111000010
011110111100000001111111	10011101111000001001000100

s349f_2

Input (24 bits)	Output (26 bits)
101010101010101011111111	10101010101010101101010101
010111100000001110000000	00011110000000101011110000
111110000011110001111111	00011100000111010001111100
111000011100011000000000	00001101111001110010001111
011110111100000001111111	10011101111000001010000100

Reproducibility: Run `python p1-1.py` in the provided environment; it builds each circuit from `./files/*.txt` and prints the vectors used to populate the tables above.